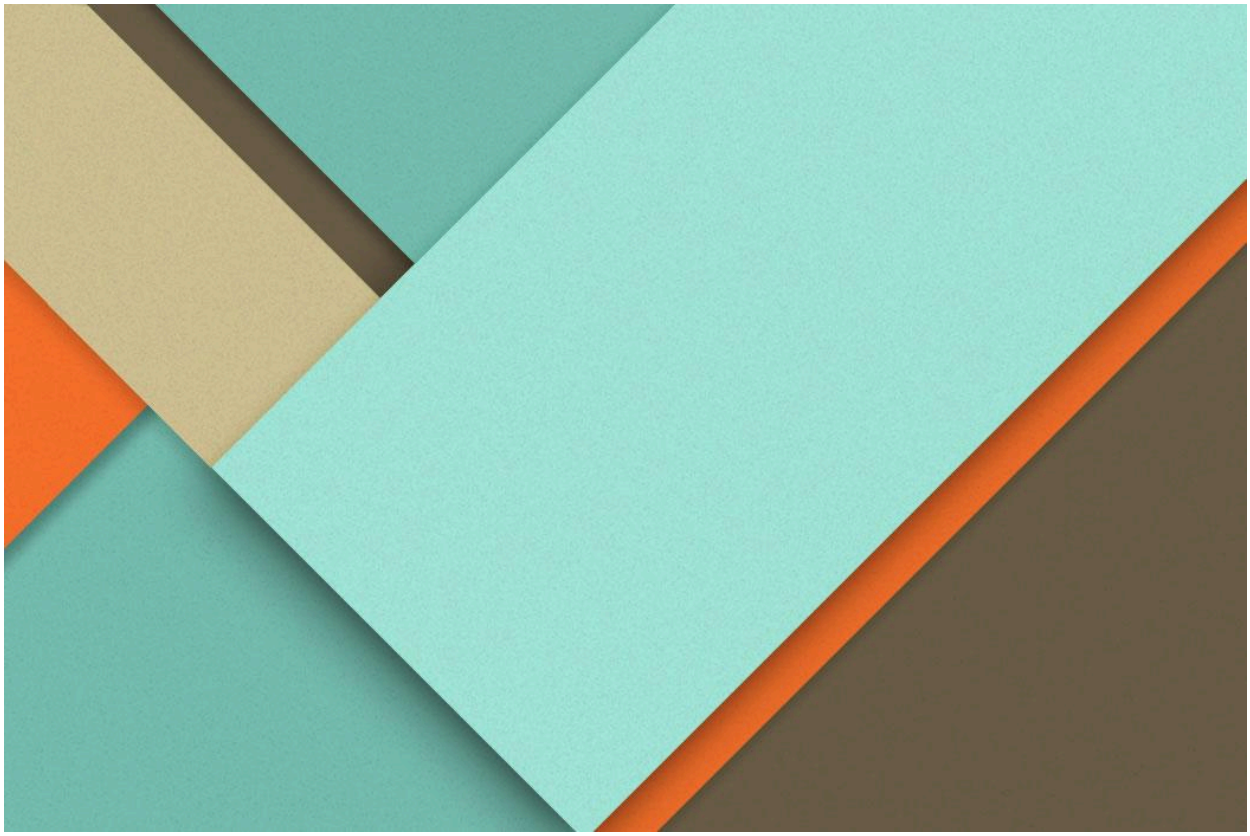


Stage 1 Submission

Conner Lavineway - 7941006
Ali Rajabi Shakib - 7966878
Paschal Olisa - 7879350

Overview

We decided to use the F1 Dataset from the recommended datasets. It provides relatively complete and non-trivial tables that go beyond simple record keeping and lookup; The tables in this data set are flexible enough to answer many complex questions.

Original Data

The database we chose contains the following tables:

- `races`
  - Stores each race weekend
  - Includes season/year, round, circuit, race date/time, and session dates/times for practice, qualifying, and sprint
- `circuits`
  - Stores circuit information
  - Includes circuit name, city, and country
- `drivers`
  - Stores driver information
  - Includes driver reference, number, code, first name, last name, date of birth, and nationality
- `teams`
  - Stores team information
  - Includes team reference, name, and nationality
- `results`
  - Stores Grand Prix race results for each driver in each race
  - Includes team, car number, grid position, finishing position, points, laps, race time, fastest lap data, and status
- `sprint_results`
  - Stores sprint race results for each driver in each sprint weekend
  - Includes team, car number, grid position, finishing position, points, laps, time, fastest lap data, and status
- `qualifying`
  - Stores qualifying results for each driver in each race weekend
  - Includes team, car number, qualifying position, and Q1/Q2/Q3 times
- `lap_times`
  - Stores lap-by-lap race performance data
  - Includes race, driver, lap number, lap position, lap time, and lap time in milliseconds
- `pit_stops`
  - Stores pit stop information during races
  - Includes race, driver, stop number, lap, time of day, duration, and duration in milliseconds
- `status`
  - Stores status codes/descriptions associated with race and sprint results
  - Examples include finished, accident, mechanical failure, and disqualification
- `driver_standings`
  - Stores championship standings for drivers after each race
  - Includes points, position, and number of wins

- `team_standings`
  - Stores championship standings for teams after each race
  - Includes points, position, and number of wins
- `team_results`
  - Stores per-race summary results for teams
  - Includes the points scored by each team in each race

Summary

The database is organized into three main groups of tables:

The core entity tables—`races`, `circuits`, `drivers`, and `teams`—store the main objects in the domain.

The performance and event tables—`results`, `sprint_results`, `qualifying`, `lap_times`, and `pit_stops`—capture what happens during each race weekend and record driver performance in detail.

The summary and support tables—`status`, `driver_standings`, `team_standings`, and `team_results`—provide descriptive labels and aggregated championship or race-level summaries.

All the above are defined as SQL TABLES in the original database.

EER Diagram and Data Remodeling

The original data base had a few problems and potential pitfalls;

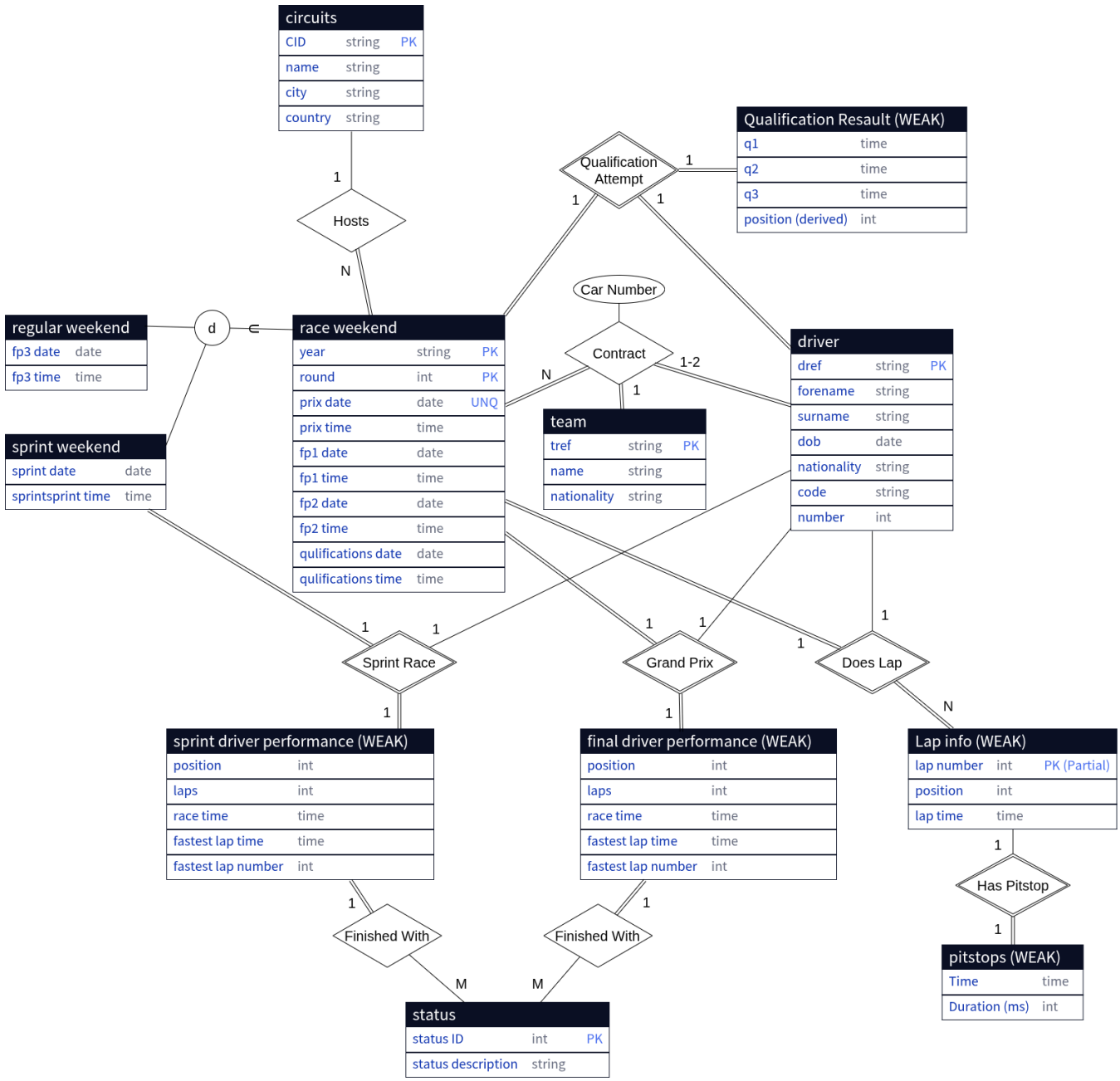
Each Table included a redundant `RowID` Attribute, even if other attributes were enough to uniquely determine a row

Many derived attributes were stored in each table directly. For example, the `lap-times` table, the time it took the driver to complete the lap was stored both in milliseconds and as a human readable string (“HH:MM:SS”).

Duplicated data is scattered across many tables. For example, the team each driver plays for is stored in `pit_stops`, `results`, `sprint_results`, and `qualification` tables even though it logically never changes for the duration of the race.

There exists entire tables like `driver_standing` and `team_standing` where all the data in them can be easily derived from data in other tables.

As an attempt to solve the above issues we created the following remodel for our data



Cardinality and Constraints

Race Weekend has 2 disjoint subsets; regular weekend, which is a normal race weekend where drivers do 3 free practice sessions on the circuit without it affecting their score or qualification, And sprint weekends where instead of the third free practice, drivers do a small race with bonus points for the top standing.

- **Hosts:**
    - **1:N** Between *Circuits* and *Race Weekends*
        - A race weekend is held at only one circuit
        - A circuit can host many race weekends over time
    - ***Circuits* participates partially** because a circuit entity can exist in the database without hosting any races (rare but possible)
    - ***RaceWeekend* participates totally** because a race **must** be held at some host circuit.

- **Does Lap**
    - **1:1:N** between driver, race weekend and lap info
        - The combination {R, L}, where R and L are PK of a Race and lap info respectively, Can occur in this relation with at most one race. This is because the PK of lapinfo determines the driver that did the lap. Hence why Drivers has a cardinality of 1.
        - Similar argument like above; lapinfo.PK -> race.PK so race has cardinality of 1
        - A driver can do many laps in the same race. Hence lap has cardinality of N
    - ***Race Weekend* participates totally** because every race must have someone doing some lap in it.
    - ***Lap Info* participates totally** because all laps occurred at some race by some driver

    - **Driver participates partially** because not every driver gets the chance to do a lap at the grand prix. Some may never qualify.
- **Has Pitstop**
    - **1:1** Between *Pit Stops* and *Lap Info*,
    - ***Pit Stops* participates totally** because all pit stops must happen during a lap
    - ***Lap Info* participates partially** because you can have a lap with no pit stops
- **Contract**
    - **1-2:N:1** Driver, Race Weekend, and Teams,
        - Each team in a race weekend can recruit at most two drivers
        - The same driver in the same team can be in many (N) race weekends
        - Drivers in a race weekend can be exactly with one team
    - ***Driver* participates totally** because to participate in a race weekend all drivers need a team
    - ***Teams* participates totally** because to participate in a race weekend all teams need a driver
    - ***Race Weekend* participates totally** because for a race weekend to be held you need drivers and teams.
- **Qualification Attempts**
    - **1:1:1** between *Race Weekend*, *Drivers* and *Qualification*,
        - Every driver has exactly 1 qualification result for each race entry they are in.
        - The PK of qualification determines driver and race weekend they were in
    - ***Race Weekend* participates totally** because every race must have someone trying to qualify for it
    - ***Drivers* participate totally** because we don't store drivers that have never even attempted to qualify
    - ***Qualification* participates totally** because every qualification must belong to a race and a driver (its identifying parents)
- **Grand Prix**
    - **1:1:1** between *Race Weekend*, *Final Driver Performance* and *Driver*,

- ***Race Weekend*** **participates totally once** per entity because every race regardless of subset affects Final Driver Performance and each race can only be ran once
- ***Final Driver Performance*** **participates totally once** for the same reasons as Race Weekend, Final Driver Performance is a weak entity because without race weekends the data doesn't make sense.
- ***Drivers*** **participate partially once** as only drivers that qualify to race have their stats affected and each race weekend only affects each driver once

- **Sprint Race**
  - **1:1:1** between *Sprint Weekend*, *Drivers* and *Sprint Driver Performance*
  - ***Sprint Weekend*** **participates totally once** because all sprints affect the Sprint Driver Performance and can only be ran once
  - ***Sprint Driver Performance*** **participates totally once** for the same reasons as Sprint Weekend. Sprint Driver Performance is a weak entity because without race weekends the data doesn't make sense.
  - ***Drivers*** **participates partially once** as only a drivers that qualify to race have their stats affected and drivers can only race once

- **Finished With**
  - describes 2 relationships both, one between *Sprint Driver Performance* and *Status* the other between *Final Driver Performance* and *Status* both are functionally the same relationship but using different tables
  - **1:M**
  - ***Status*** **partially participates** because both Sprint Driver Performance and Final Driver Performance have a status but not necessarily all statuses are used
  - ***Final Driver Performance*** **and** ***Sprint Driver Performance*** **totally participate** because every race finishes with exactly 1 status

Relational Model

1. RACE_WEEKEND(*year*, *round*, *circuit, prix_date, prix_time, fp1_date, fp1_time, qual_date, qual_time)*
   1.1. SPRINT_WEEKEND*(year, round, sprint_date, sprint_time)*
   1.2. REGULAR_WEEKEND*(year, round, fp3_date, fp3_time)*

   Comments:

   - Separating into above relations (as opposed to keeping *fp3* and *sprint* information in one relation with RACE_WEEKEND) minimizes null values and is more natural because subsets are distinct.
   - *{year ,round}* is the inherited key in SPRINT_WEEKEND and REGULAR_WEEKEND. It is a foreign key referencing the primary key of RACE_WEEKEND.
2. CIRCUITS(circuitID, name, city, country)
3. DRIVERS(driverRef, name, nationality)
4. TEAMS(teamRef, name, nationality)
5. STATUS(statusID, description)
6. QUAL_RESULT(*year*, *round*, *driverRef*, *q1*, *q2*, *q3*)

   Comments:

   - The attribute *position* can be computed by looking at other rows with the same {*year, round*} value and comparing their qualification times (q1,q2,q3).
   - {year, round} is a foreign key referencing the PK of RACE_WEEKEND.
   - {driverRef} is a foreign key referencing the PK of DRIVER.
7. SPRINT_PERF(year, round, driverRef, racetime, lapNum, fLapTime, fLapNum, statusID)

   Comments:

   - {year, round} is a foreign key referencing the PK of SPRINT_WEEKEND; if it has a value that exists in RACE_WEEKEND but not in SPRINT_WEEKEND, an error must arise.
   - {driverRef} is a foreign key referencing the PK of DRIVER.
   - {statusID} is a foreign key referencing the PK of STATUS.
8. PRIX_PERF(year, round, driverRef, racetime, numLaps, fLapTime, fLapNum, statusID)

   Comments:

   - {year, round} is a foreign key referencing the PK of RACE_WEEKEND.
   - {driverRef} is a foreign key referencing the PK of DRIVER.

- {statusID} is a foreign key referencing the PK of `STATUS`.

9. `LAP_INFO`(year, round, driverRef, lapNum, position, lap_time)

   Comments:

   - {year, round} is a foreign key referencing the PK of `RACE_WEEKEND`.
   - {driverRef} is a foreign key referencing the PK of `DRIVER`.
   - The combination {year, round, driverRef} is not unique and needs {lapNum} to form a key.

10. `PIT_STOP`(year, round, driverRef, lapNum, time , duration)

    Comments:

    - {year, round, driverRef, lapNum} is a foreign key referencing the PK of `LAP_INFO`.

11. `CONTRACT`(driverRef, year, round, teamRef, car_number)

    Comments:

    - {year, round} is a foreign key referencing the PK of `RACE_WEEKEND`.
    - {driverRef} is a foreign key referencing the PK of `DRIVER`.
    - {TeamRef} is a foreign key referencing the PK of `TEAMS`.

SQL version of the above is in the appendix with more detail.

Comments on Normalization

All the above tables are in BCNF. While remodelling the original data, we subconsciously grouped every functional dependency into its own table. We assert that, in every relation, the only nontrivial functional dependencies are of the form “superkey determines all other attributes.” A relation is in BCNF exactly when every nontrivial functional dependency has a superkey on the left-hand side. Since this is true for every relation in the schema, every relation satisfies the definition of BCNF. Therefore, the entire schema is in BCNF.

The sole non-trivial dependency is the following dependency in the `RACE_WEEKEND` relation:

{prix_date} -> {year, round, circuit, prix_time, fp1_date, fp1_time, qual_date, qual_time}

Which makes *prix_date* an alternative candidate key and does not break BCNF.

Appendix A: SQL schema of our model

```sql
CREATE TABLE IF NOT EXISTS CIRCUITS (
    circuitID   INTEGER PRIMARY KEY,
    name        TEXT NOT NULL,
    city        TEXT NOT NULL,
    country     TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS DRIVERS (
    driverRef   TEXT PRIMARY KEY,
    name        TEXT NOT NULL,
    nationality TEXT
);

CREATE TABLE IF NOT EXISTS TEAMS (
    teamRef     TEXT PRIMARY KEY,
    name        TEXT NOT NULL,
    nationality TEXT
);

CREATE TABLE IF NOT EXISTS STATUS (
    statusID     INTEGER PRIMARY KEY,
    description  TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS RACE_WEEKEND (
    year        INTEGER NOT NULL,
    round       INTEGER NOT NULL,
    circuitID   INTEGER NOT NULL,
    prix_date   TEXT NOT NULL,
    prix_time   TEXT,
    fp1_date    TEXT,
    fp1_time    TEXT,
    qual_date   TEXT,
    qual_time   TEXT,
    PRIMARY KEY (year, round),
    FOREIGN KEY (circuitID) REFERENCES CIRCUITS(circuitID)
);
```

```sql
CREATE TABLE IF NOT EXISTS SPRINT_WEEKEND (
    year         INTEGER NOT NULL,
    round        INTEGER NOT NULL,
    sprint_date  TEXT NOT NULL,
    sprint_time  TEXT,
    PRIMARY KEY (year, round),
    FOREIGN KEY (year, round) REFERENCES RACE_WEEKEND(year, round)
);

CREATE TABLE IF NOT EXISTS REGULAR_WEEKEND (
    year      INTEGER NOT NULL,
    round     INTEGER NOT NULL,
    fp3_date  TEXT NOT NULL,
    fp3_time  TEXT,
    PRIMARY KEY (year, round),
    FOREIGN KEY (year, round) REFERENCES RACE_WEEKEND(year, round)
);

CREATE TABLE IF NOT EXISTS QUAL_RESULT (
    year       INTEGER NOT NULL,
    round      INTEGER NOT NULL,
    driverRef  TEXT NOT NULL,
    q1         TEXT,
    q2         TEXT,
    q3         TEXT,
    PRIMARY KEY (year, round, driverRef),
    FOREIGN KEY (year, round) REFERENCES RACE_WEEKEND(year, round),
    FOREIGN KEY (driverRef) REFERENCES DRIVERS(driverRef)
    -- position omitted because it is derivable
);

CREATE TABLE IF NOT EXISTS SPRINT_PERF (
    year       INTEGER NOT NULL,
    round      INTEGER NOT NULL,
    driverRef  TEXT NOT NULL,
    racetime   TEXT,
    lapNum     INTEGER,
    fLapTime   TEXT,
    fLapNum    INTEGER,
    statusID   INTEGER NOT NULL,
    PRIMARY KEY (year, round, driverRef),
    FOREIGN KEY (year, round) REFERENCES SPRINT_WEEKEND(year, round),
```

```sql
    FOREIGN KEY (driverRef) REFERENCES DRIVERS(driverRef),
    FOREIGN KEY (statusID) REFERENCES STATUS(statusID)
);

CREATE TABLE IF NOT EXISTS PRIX_PERF (
    year       INTEGER NOT NULL,
    round      INTEGER NOT NULL,
    driverRef  TEXT NOT NULL,
    racetime   TEXT,
    lapNum     INTEGER,
    fLapTime   TEXT,
    fLapNum    INTEGER,
    statusID   INTEGER NOT NULL,
    PRIMARY KEY (year, round, driverRef),
    FOREIGN KEY (year, round) REFERENCES RACE_WEEKEND(year, round),
    FOREIGN KEY (driverRef) REFERENCES DRIVERS(driverRef),
    FOREIGN KEY (statusID) REFERENCES STATUS(statusID)
);

CREATE TABLE IF NOT EXISTS LAP_INFO (
    year       INTEGER NOT NULL,
    round      INTEGER NOT NULL,
    driverRef  TEXT NOT NULL,
    lapNum     INTEGER NOT NULL,
    position   INTEGER,
    lap_time   TEXT,
    PRIMARY KEY (year, round, driverRef, lapNum),
    FOREIGN KEY (year, round) REFERENCES RACE_WEEKEND(year, round),
    FOREIGN KEY (driverRef) REFERENCES DRIVERS(driverRef)
);

CREATE TABLE IF NOT EXISTS PIT_STOP (
    year       INTEGER NOT NULL,
    round      INTEGER NOT NULL,
    driverRef  TEXT NOT NULL,
    lapNum     INTEGER NOT NULL,
    time       TEXT,
    duration   REAL,
    PRIMARY KEY (year, round, driverRef, lapNum),
    FOREIGN KEY (year, round, driverRef, lapNum)
        REFERENCES LAP_INFO(year, round, driverRef, lapNum)
);
```